



Actividad 4

Entrega

- **Fecha y hora:** Miércoles 15 de octubre del 2025, a las 22:00
- **Lugar:** repositorio personal, rama main.
- **Github classroom:** [link a repositorio personal](#)

Objetivo

- **Modificar** una interfaz para que permita iniciar sesión, refrescar un token o cerrar sesión.
- **Comprender** el flujo de autenticación utilizando JWT (JSON Web Tokens).
- **Implementar** el manejo seguro de una sesión mediante cookies.

1. Descripción

Durante esta actividad se deberá desarrollar una aplicación en React que implemente un sistema de autenticación mediante JWT (JSON Web Tokens), utilizando la API pública de DummyJSON.

La aplicación deberá cumplir con las siguientes funcionalidades principales:

- Realizar una solicitud POST para autenticar al usuario mediante credenciales válidas.
- Obtener y almacenar el **token JWT** dentro de una **cookie**, permitiendo su persistencia en el navegador.
- Implementar la posibilidad de **modificar, refrescar y eliminar** el token según el estado de la sesión del usuario.

2. Instrucciones

2.1. Archivo: login.tsx

1. Realiza una solicitud POST a la API pública <https://dummyjson.com/auth/login>, enviando las credenciales del usuario (username y password) junto con el parámetro `expiresInMins`.
2. Implementa un formulario de inicio de sesión con los siguientes campos:
 - username
 - password
 - Opción Remember Me (checkbox)

3. Utiliza alguno de los siguientes usuarios de prueba proporcionados por la API:

Username	Password	Nombre
emilys	emilyspass	Emily Johnson
michaelw	michaelwpass	Michael Williams
sophiab	sophiabpass	Sophia Brown
jamesd	jamesdpass	James Davis

4. Modifica la función `handleSubmit` para incluir el siguiente flujo:

- Inicia el estado de carga y limpia errores previos antes de realizar la solicitud.
- Envía las credenciales a la API mediante `axios.post()`.
- Si la autenticación es exitosa:
 - Extrae el `accessToken` de la respuesta.
 - Decodifica el token con `jwt-decode` para obtener la fecha de expiración.
 - Si el usuario seleccionó `Remember Me`, almacena el token en una cookie persistente con su fecha de expiración.
 - Si no se seleccionó, guarda el token en una cookie de sesión temporal.
 - Redirige automáticamente al usuario a la vista `/user`.
- Si ocurre un error en la autenticación:
 - Muestra un mensaje de error en pantalla.

5. No modifiques el `useEffect` que se dejó en el código base. Este simplemente redirige a `/user` en caso de que exista un token válido.

2.2. Archivo: `user.tsx`

1. Lectura de sesión

- Al cargar la vista `/user`, lee el token almacenado en la cookie `token` usando la librería `js-cookie`.
- Si el token existe y no ha expirado, decodifícalo con `jwt-decode` y guárdalo en el estado local.
- Implementa un hook `useEffect(() => {}, [])` que se ejecute una sola vez al montar el componente, para inicializar el token y cargar los datos del usuario si existe una sesión activa.

2. Contador de expiración

- Implementa un segundo hook `useEffect(() => {}, [token])` que decodifique el token JWT y calcule en tiempo real el tiempo restante hasta su expiración.
- El contador debe actualizarse cada segundo y mostrar el tiempo en formato `hh:mm:ss`.
- Cuando el token expire, se debe detener el contador y mostrar el texto **“Expirado”**.

3. Carga de datos del usuario

- Implementa una función `fetchUserData()` que haga una solicitud GET a `https://dummyjson.com/auth/me`.
- Incluye el token JWT en los encabezados de la solicitud usando el formato: `Authorization: Bearer <token>`.
- Muestra en la interfaz los datos básicos del usuario autenticado (ID, nombre, email, username, género, token, etc.).

- Controla los estados de carga y error durante la obtención de datos.

4. Refresco del token

- Crea una función `refreshToken()` que realice una solicitud POST a `https://dummyjson.com/auth/refresh`.
- Envía el token actual y el número de minutos que deseas tener de validez en el token.
- Si la solicitud es exitosa:
 - Decodifica el nuevo token y obtiene su nueva fecha de expiración.
 - Guarda el nuevo token en cookies y actualiza el estado local.
 - Vuelve a ejecutar `fetchUserData()` para actualizar los datos del usuario.

5. Cerrar sesión

- Implementa una función `logout()` que elimine las cookies relacionadas con la sesión (`token`, `refreshToken`).
- Limpia los estados locales del usuario y redirige a la página principal (/).

3. Consideraciones Importantes

- Puedes usar `https://jwt.io/` para explorar y entender la información contenida en los tokens.
- Utiliza `console.log()` y las herramientas de desarrollador del navegador (DevTools) para verificar las cookies y el flujo de autenticación.
- Asegúrate de mantener un flujo ordenado y manejar correctamente los estados de carga y error en cada función.
- Revisa la responsividad de la interfaz en diferentes tamaños de pantalla antes de entregar la actividad.

4. Requisitos

A continuación, se detallan los requisitos que debes cumplir para obtener el puntaje máximo en esta actividad.

1. Al iniciar sesión con credenciales válidas, realizando POST a `https://dummyjson.com/auth/login`, se redirige a vista `/user`.
2. En caso de error en iniciar sesión, se muestra mensaje de error al ingresar credenciales inválidas.
3. Al ingresar correctamente se guarda el token (retornado por la API) en una cookie llamada `'token'` con fecha de expiración (`tokenDecoded.exp * 1000`).
4. En la vista `/user` se muestran los datos del usuario: ID, nombre, email, username, género y token (luego de hacer GET a `https://dummyjson.com/auth/me` con el header `Authorization: Bearer <token>`).
5. Se muestra temporizador que indica la expiración del token y se actualiza en caso de refrescar el token.
6. Se refresca el token al actualizar el tiempo de expiración (tanto en la cookie como en la vista `/user`).
7. Si se inicia sesión con `'Remember Me'` activado, entonces al cerrar y volver a abrir el navegador, el usuario seguirá logeado siempre que el token siga siendo válido.
8. Si se inicia sesión con `'Remember Me'` desactivado, entonces al cerrar y volver a abrir el navegador, se abrirá directamente el login (sin dejar rastro de algún token.). Esto porque se generó una cookie de sesión temporal.

9. Si el token expira, se debe mostrar en la vista user el mensaje de 'Expirado' en el temporizador.
10. Si el token expiró, no se debe poder refrescar el token, mostrando un mensaje de error al presionar 'Refrescar Token'.
11. La funcionalidad 'Cerrar sesión' elimina el token de las cookies, limpia el estado y redirige a '/'.

Para que un requisito se considere como “cumplido”, deberás seguir estrictamente lo que se indica en su descripción.

Destacar que muchas de las funcionalidades ya se encuentran implementadas a nivel de código. Simplemente debes asegurarte de que todo funcione correctamente y seguir las instrucciones.

5. Asignación de Puntaje

El puntaje que obtendrás en esta actividad se calculará a partir del porcentaje de requisitos que hayas cumplido. En la siguiente tabla, podrás observar la distribución del puntaje.

Puntaje	Porcentaje de requisitos logrados	Logro
1 punto	100 %	Completamente logrado
0.5 puntos	50 % a 99 %	Parcialmente logrado
0 puntos	<50 %	No logrado